Type Classes via Dictionaries

JSON encoder + Show/Eq for domain records and composites

Season 7 · Functional · Advanced

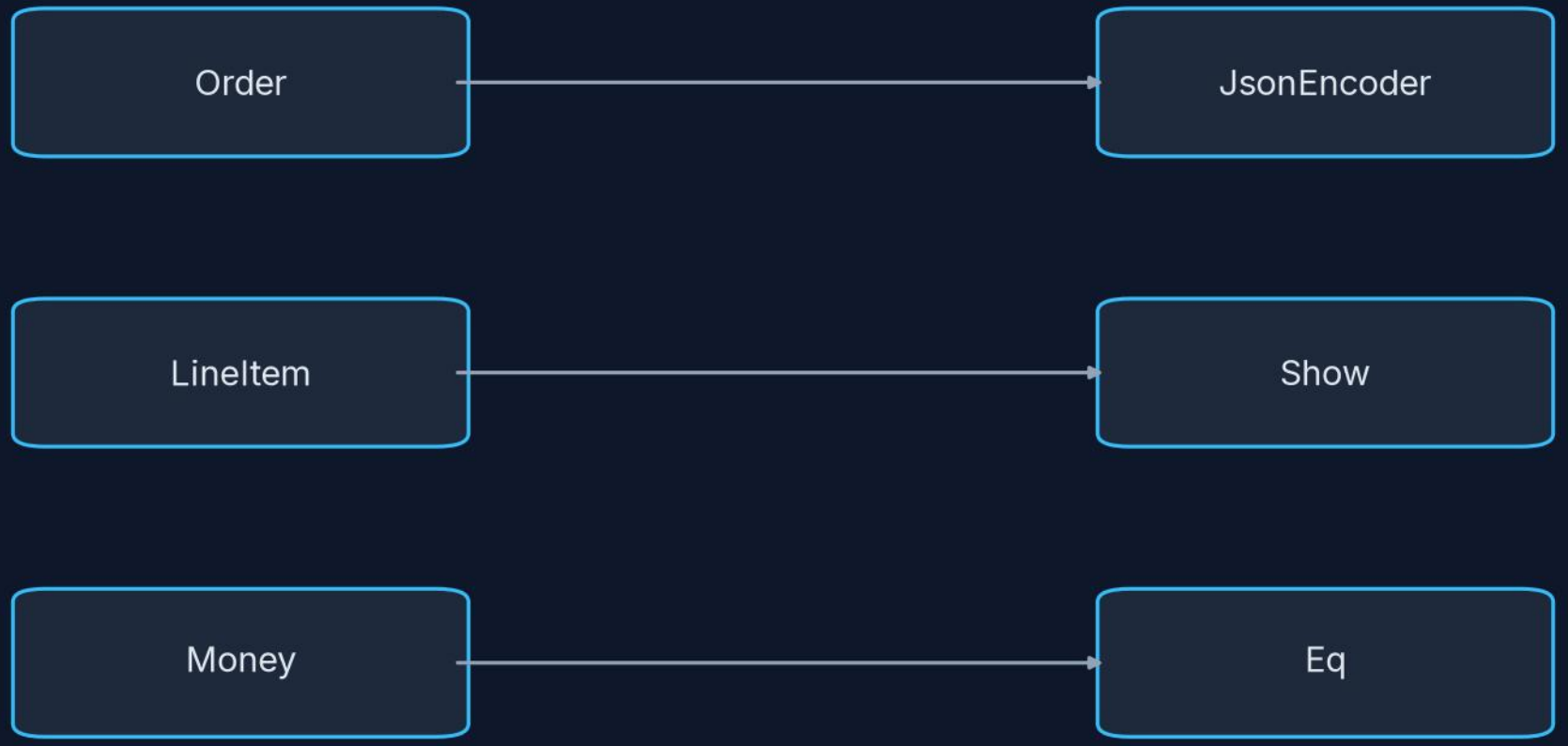
Today's Hook

- Problem: JSON encoder + Show/Eq for domain records and composites
- Naive pain cue: Failure path is asserted in ProblemTest (injected fault / invalid input)
- We will refactor to Type Classes with a runnable Demo.
- Failure path is locked by ProblemTest.

The Problem

- Scenario: JSON encoder + Show/Eq for domain records and composites
- Constraints: solve «JSON encoder + Show/Eq for domain records and composites» with Type Classes — keep the demo runnable and the failure path observable.
- Naive pain: Naive: ad-hoc toString/JSON in every call site — no reuse, easy to drift.
- Success looks like: json={"id":"o1","items":[{"sku":"SKU1","qty":2,"pr ice":{"currency":"USD","cents":1999}},{"sku":"SKU2","qty":1,"price":{"

Structure



Type Classes: JSON encoder + Show/Eq for domain records and composites

Real-World Scenario

- Running example: JSON encoder + Show/Eq for domain records and composites
- Key types: dictionaries, TypeClasses, Money, LineItem
- Happy-path stdout cue: json={"id":"o1","items":[{"sku":"SKU1","qty":2,"price":{"currency":"USD","cents":1999}},{"sku":"SKU2","qty":1,"price":{"
- Failure-path stdout cue: Failure path is asserted in ProblemTest (injected fault / invalid input)

Naive Code

```java
package com.buildwithwaqar.patterns.functional.typeclass.problem;

/** Naive: ad-hoc toString/JSON in every call site — no reuse, easy to drift. */
public final class NaiveJson {
    public static String moneyJson(TypeClasses.Money m) {
        return "{cents:" + m.cents() + "}"; // forgot currency
    }
    public static String orderJson(TypeClasses.Order o) {
        return "{id:" + o.id() + "}"; // forgot items
    }
}
```

Why Naive Hurts

- Naive: ad-hoc toString/JSON in every call site — no reuse, easy to drift.
- Without Type Classes, call sites absorb the wrong details.
- Failure mode to show on camera: Failure path is asserted in ProblemTest (injected fault / invalid input)
- New requirements force shotgun edits across the naive code.

Solution

```java
package com.buildwithwaqar.patterns.functional.typeclass.problem;

import java.util.List;
import java.util.Objects;
import java.util.stream.Collectors;

/** Explicit type-class dictionaries: Show, Eq, JsonEncoder for domain records. */
public final class TypeClasses {
    private TypeClasses() {}

    public record Money(String currency, int cents) {}
    public record LineItem(String sku, int qty, Money price) {}
    public record Order(String id, List<LineItem> items) {}

    public interface Show<A> { String show(A a); }
    public interface Eq<A> { boolean eqv(A a, A b); }
    public interface JsonEncoder<A> { String encode(A a); }

    public static final Show<Money> showMoney = m -> m.currency() + " " + (m.cents() / 100.0);
    public static final Eq<Money> eqMoney = (a, b) ->
            Objects.equals(a.currency(), b.currency()) && a.cents() == b.cents();
    public static final JsonEncoder<Money> jsonMoney = m ->
            "{\"currency\":\"" + m.currency() + "\",\"cents\":" + m.cents() + "}";
```

Solution (2)

```
    public static final Show<LineItem> showLine = li ->
            li.qty() + "x" + li.sku() + "@" + showMoney.show(li.price());
    public static final JsonEncoder<LineItem> jsonLine = li ->
            "{\"sku\":\"" + li.sku() + "\",\"qty\":" + li.qty() + ",\"price\":" + jsonMoney.encode(li.price()) + "}";

    /** Derivation for composites. */
    public static final JsonEncoder<Order> jsonOrder = o -> {
        String items = o.items().stream().map(jsonLine::encode).collect(Collectors.joining(","));
        return "{\"id\":\"" + o.id() + "\",\"items\":[" + items + "]}";
    };
    public static final Show<Order> showOrder = o ->
            "Order(" + o.id() + ": " + o.items().stream().map(showLine::show).collect(Collectors.joining(", ")) + ")";
    public static final Eq<Order> eqOrder = (a, b) ->
            Objects.equals(a.id(), b.id()) && a.items().equals(b.items());
}
```

Runnable main

```
package com.buildwithwaqar.patterns.functional.typeclass.problem;

import java.util.List;

public final class ProblemDemo {
    public static void main(String[] args) {
        System.out.println("=== Type Classes: Show / Eq / JsonEncoder ===");
        var m = new TypeClasses.Money("USD", 1999);
        var order = new TypeClasses.Order("o1", List.of(
                new TypeClasses.LineItem("SKU1", 2, m),
                new TypeClasses.LineItem("SKU2", 1, new TypeClasses.Money("USD", 500))));
        System.out.println("--- naive (incomplete JSON) ---");
        System.out.println(NaiveJson.moneyJson(m));
        System.out.println(NaiveJson.orderJson(order));
        System.out.println("--- type-class dictionaries ---");
        System.out.println("show=" + TypeClasses.showOrder.show(order));
        System.out.println("json=" + TypeClasses.jsonOrder.encode(order));
        System.out.println("eqSelf=" + TypeClasses.eqOrder.eqv(order, order));
        System.out.println("eqOther=" + TypeClasses.eqOrder.eqv(order, new TypeClasses.Order("o2", List.of())));
    }
}
```

Walkthrough Output

```
=== Type Classes: Show / Eq / JsonEncoder ===
--- naive (incomplete JSON) ---
{cents:1999}
{id:o1}
--- type-class dictionaries ---
show=Order(o1: 2xSKU1@USD 19.99, 1xSKU2@USD 5.0)
json={"id":"o1","items":[{"sku":"SKU1","qty":2,"price":{"currency":"USD","cents":1999}},{"sku":"SKU2","qty":1,"price":{"currency":"USD","cents":500}}]}
eqSelf=true
eqOther=false
```

Step-by-Step Trace

- Match stdout to stages — look for: json={"id":"o1","items":[{"sku":"SKU1","qty":2,"price":{"currency":"USD","cents":1999}},{"sku":"SKU2","qty":1,"price":{"
- Then the injected failure: Failure path is asserted in ProblemTest (injected fault / invalid input)
- Re-run: java -cp ... 61-type-classes ProblemDemo (see episode folder).
- Pause the video and predict the next printed line.

Modern Java 25 / Zazr

```java
package com.buildwithwaqar.patterns.functional.typeclass.problem;

import java.util.List;
import java.util.Objects;
import java.util.stream.Collectors;

/** Explicit type-class dictionaries: Show, Eq, JsonEncoder for domain records. */
public final class TypeClasses {
    private TypeClasses() {}

    public record Money(String currency, int cents) {}
    public record LineItem(String sku, int qty, Money price) {}
    public record Order(String id, List<LineItem> items) {}

    public interface Show<A> { String show(A a); }
    public interface Eq<A> { boolean eqv(A a, A b); }
    public interface JsonEncoder<A> { String encode(A a); }

    public static final Show<Money> showMoney = m -> m.currency() + " " + (m.cents() / 100.0);
    public static final Eq<Money> eqMoney = (a, b) ->
            Objects.equals(a.currency(), b.currency()) && a.cents() == b.cents();
    public static final JsonEncoder<Money> jsonMoney = m ->
            "{\"currency\":\"" + m.currency() + "\",\"cents\":" + m.cents() + "}";
```

Modern Java 25 / Zazr (2)

```
    public static final Show<LineItem> showLine = li ->
            li.qty() + "x" + li.sku() + "@" + showMoney.show(li.price());
    public static final JsonEncoder<LineItem> json
```

Trade-offs

- Use Type Classes when «JSON encoder + Show/Eq for domain records and composites» repeats and a simpler feature does not erase it.
- Refuse extra types when a sealed switch or a function value already suffices.
- Teaching demos omit production metrics, retries, and multi-region clocks.
- Interview soundbite: state the force behind Type Classes in product language.

Related Patterns

- Name the pattern usually confused with Type Classes and separate the forces.
- Sketch the roles for «JSON encoder + Show/Eq for domain records and composites» from memory in sixty seconds.
- Point to the next episode in this season on the playlist map.
- Link this episode's series/ folder in the YouTube description.

Recap

- Problem: JSON encoder + Show/Eq for domain records and composites
- Naive pain → Type Classes boundary → Demo stdout.
- Remember the failure cue: Failure path is asserted in ProblemTest (injected fault / invalid input)
- Break a ProblemTest assertion, watch it fail, restore it.